

Evolved Neural Controllers vs Hand-Designed Rules for Stigmergic Sorting

Abstract

1 Introduction

Collective behaviour in social insects presents a puzzle that has resisted simple explanation for over a century: how do individuals with no global view, no central coordinator, and no shared plan produce outcomes that look designed? A colony of ants sorting its brood into colour-coded piles seems to *know* something that no single ant knows. The answer, as Grassé [7] observed, lies not in the agents but in the environment they leave behind — each moved pellet, each deposited pheromone trace, becomes a signal that biases the next action. The work is distributed across time and space, embedded in the world itself. This is stigmergy.

What attracted us to this problem is not that it is well understood, but that it sits at an unresolved tension between two very different ways of engineering collective behaviour. On one side stand hand-designed rules like Deneubourg’s probabilistic pickup/drop model [3]: elegant, minimal, provably effective for a narrow class of tasks. On the other stand evolved neural controllers: opaque, data-hungry, but capable of discovering strategies that no human would specify. The question is not which approach is “better” in the abstract — it is which one works better *here*, for this task, under these constraints, and what the answer tells us about the relationship between simplicity and learned complexity in collective systems.

We set out to test a specific hypothesis: that evolved neural controllers can discover sorting strategies that outperform the canonical Deneubourg baseline, but only under certain environmental conditions. Our intuition comes from two observations. First, the Deneubourg rule is remarkably efficient — it sorts 200 pellets into coherent clusters using nothing but local similarity counts and two parameters. Beating it is not trivial. Second, the rule is also rigid — it cannot adapt its behaviour based on what an agent is currently carrying, or modulate its exploration strategy in response to global progress. A neural controller with access to richer state information might exploit these blind spots, but only if the evolutionary budget is sufficient to discover the relevant structure.

We do not expect a single answer. If evolution beats the baseline, the finding is that learned controllers can extract value from state-dependent action selection in collective tasks — a result that generalises to other swarm robotics problems. If the baseline holds or wins, the finding is equally interesting: that minimal hand-designed rules with good parameter choice remain highly efficient, and that evolution under tight computational budgets struggles to discover them. This would be a direct empirical line back to Brooks’s argument for “intelligence without representation” [1] — that embodied, reactive controllers can outperform learned policies when the environment provides sufficient structure.

Either outcome is informative. The boundary between them — the regime in which each approach dominates — is the finding we are after.

2 Background

2.1 Stigmergic Sorting

The term *stigmergy* was coined by Grassé [7] to describe indirect coordination among social insects through environmental modifications. Workers leave traces (pheromones, moved objects) that stimulate subsequent actions by themselves or neighbours, producing emergent collective behaviour without central control.

Deneubourg et al. [3] formalised this into a computational sorting model: mobile agents pick up items probabilistically based on local similarity, carrying them until the surrounding environment matches the carried item’s type. The pickup probability decreases with the number of similar neighbours (isolated items are preferred), while the drop probability increases with local similarity (items are deposited near matching clusters). Over time, this simple rule produces globally sorted configurations from random initial placement.

The model was motivated by observations of ant brood sorting [5], where worker ants organise nymphs into colour-coded piles using only local cues. This biological grounding makes stigmergic sorting a natural testbed for comparing hand-designed behavioural rules against evolved controllers: the task is simple enough that minimal rules succeed, yet complex enough that learned policies might discover superior strategies.

2.2 Neuroevolution in Collective Tasks

3 Methods

3.1 Environment

We model a two-dimensional continuous arena of 100×100 units with hard-wall boundaries. The environment contains 200 pellets—100 red and 100 blue—placed uniformly at random. Twenty agents navigate the arena, each equipped with a circular sensor of radius $r = 5$ that reports the count of pellets per colour within range.

The action space is discrete with five options: *move* forward one unit in the current heading, *turn left* (-30°), *turn right* ($+30^\circ$), *pick up* a nearby pellet (within distance 1.0), and *drop* a carried pellet. Each agent can carry at most one pellet at a time.

We chose a 100×100 continuous arena rather than the 30×30 discrete grid used in the original Deneubourg formulation, because the larger space gives agents room for meaningful cluster formation without trivially saturating the environment. The discrete five-action space was selected over continuous heading control to keep the neural network output simple (five-way softmax) and the genetic algorithm search space well-behaved.

Simulation proceeds in synchronous discrete time: all agents compute actions based on the state at tick t , then all positions update simultaneously to $t+1$. Episodes run for at most 10,000 ticks, with early termination if cluster purity plateaus for 2,000 consecutive ticks ($\Delta < 0.01$).

All randomness is deterministic and reproducible. A master seed is split into independent subsystem streams (pellet placement, agent initialization, baseline stochasticity, genetic algorithm) via a `SeedBank` class. No global random state is used anywhere in the codebase.

3.2 Baseline: Deneubourg Model

Our baseline agent implements Deneubourg’s probabilistic pickup/drop rules [3]. Each agent carries at most one pellet and decides its action each tick based on local sensor readings.

When *not carrying* a pellet, the agent computes f as the fraction of the dominant colour among pellets within sensor radius r . The probability of attempting pickup is:

$$P_{\text{pickup}} = \left(\frac{k_1}{k_1 + f} \right)^2$$

When f is high (homogeneous neighbourhood), pickup is unlikely — the agent avoids breaking up existing clusters. When f is low (mixed neighbourhood), pickup is more likely. If pickup does not trigger, the agent performs a random walk (MOVE, TURN_LEFT, or TURN_RIGHT with equal probability).

When *carrying* a pellet, the agent computes f as the fraction of *similar-coloured* pellets in the sensor window. The probability of dropping is:

$$P_{\text{drop}} = \left(\frac{f}{k_2 + f} \right)^2$$

High f (many matching neighbours) increases drop probability — the agent deposits the pellet near a matching cluster. Low f suppresses dropping — the agent continues searching. If drop does not trigger, the agent random-walks.

We use $k_1 = 0.1$ and $k_2 = 0.3$, matching the starting values recommended in the original formulation. Sensor radius $r = 5$ units. These parameters were chosen as conventional defaults before any tuning; they represent the “off-the-shelf” Deneubourg configuration against which evolved controllers are compared.

3.3 Evaluation Metrics

We use two complementary metrics to quantify sorting performance.

Cluster Purity. Pellets are clustered using DBSCAN [4] with $\varepsilon = 5.0$ and `min_samples=2`, matching the agent sensor radius. For each non-noise cluster, purity is the fraction of pellets sharing the most common colour. The overall score is the arithmetic mean across all non-noise clusters, yielding a value in $[0, 1]$ where 1.0 means every cluster is monochromatic.

Cluster Count. The number of non-noise DBSCAN clusters. Lower values indicate more consolidation; however, cluster count alone is insufficient because an agent could achieve a low count by creating one large mixed cluster. Used alongside purity, it distinguishes “many pure clusters” from “few impure ones”.

Both metrics were validated against three synthetic configurations: (i) perfectly sorted (purity = 1.00, clusters = 2), (ii) uniformly random (purity ≈ 0.76 , clusters ≈ 41), and (iii) 80% clustered with 20% scattered noise (purity ≈ 0.78 , clusters ≈ 7). All values fell within $\pm 5\%$ of expected bounds.

3.4 Evolved Controller

Neural Network Architecture. Each evolved agent is controlled by a single-layer feedforward neural network with 149 weights. The network takes a 6-dimensional input vector:

$$\mathbf{x} = [\text{red_dens}, \text{blue_dens}, \text{similar_dens}, \text{dissimilar_dens}, \text{carrying_red}, \text{carrying_blue}]$$

where density values are raw pellet counts within the sensor radius, and carrying flags are binary indicators of the carried pellet’s colour. The hidden layer has 12 units with tanh activation, and the output layer has 5 units with softmax activation, producing a probability distribution over the action space {MOVE, TURN_LEFT, TURN_RIGHT, PICKUP, DROP}.

The architecture was chosen to balance expressiveness against search space tractability. A single hidden layer of 12 units provides enough capacity to learn non-trivial sensorimotor mappings (the universal approximation theorem guarantees any continuous function can be approximated with a single hidden layer [2]), while keeping the genome size (149 weights) small enough for efficient genetic search. We deliberately avoided recurrent architectures (e.g., CTRNNs) despite their biological plausibility [11], as the added complexity would require per-architecture hyperparameter tuning that confounds the baseline comparison.

Action Selection. Agents select actions greedily: the action with the highest softmax probability is executed. This deterministic policy ensures that fitness evaluations are reproducible (no stochastic variance from action sampling) and that the fitness landscape is smooth with respect to weight perturbations. We explored stochastic action selection (sampling from the softmax distribution) but found it produced a flat fitness landscape where the genetic algorithm could not discriminate between genomes [10].

Genetic Algorithm. We evolve the 149 weight vectors directly using a simple genetic algorithm [9]. The initial population of 30 genomes is generated with Xavier uniform initialization [6] for the weight matrices and zero initialization for biases. No pickup bias is applied (agents start with no innate tendency to pick up pellets), allowing the GA to discover whether pickup behaviour is beneficial.

Each generation proceeds as follows:

1. The best individual from the previous generation is preserved (elitism).
2. The remaining 29 individuals are produced by tournament selection (tournament size $k=3$) followed by per-gene Gaussian mutation ($\sigma=0.3$).
3. No crossover is used, following the observation that mutation-only GAs can be effective for neural network weight evolution when the population size is sufficient [10].

Fitness Function. The fitness function is a weighted composite of five components, each normalised to $[0, 1]$:

$$F = \alpha \cdot p_{\text{term}} + \beta \cdot p_{\text{int}} + \gamma \cdot c_{\text{consol}} + \delta_{\text{eff}} \cdot a_{\text{act}} + \varepsilon \cdot \ell_{\text{local}}$$

Terminal purity (p_{term} , $\alpha = 0.20$): cluster purity at the end of the episode. The primary sorting objective, but weighted modestly because it is a sparse signal (agents must chain wander→pickup→drop behaviours before any signal exists).

Integrated purity (p_{int} , $\beta = 0.15$): mean cluster purity sampled every 100 ticks. Provides denser gradient signal than terminal-only evaluation.

Consolidation (c_{consol} , $\gamma = 0.15$): $1 - (\text{final_clusters}/\text{initial_clusters})$. Rewards reduction in the number of DBSCAN clusters, encouraging agents to merge small clusters.

Activity (a_{act} , $\delta=0.20$): normalised count of pickup+drop events (capped at 400). Bootstrap reward that encourages agents to interact with pellets in early generations. The effective weight δ_{eff} decays linearly to zero by generation 25: $\delta_{\text{eff}} = \delta \cdot \max(0, 1 - g/25)$. This ensures activity is rewarded when the population needs it but does not dominate later generations.

Locality (ℓ_{local} , $\varepsilon=0.30$): for each drop event, the fraction of nearby pellets (within sensor radius) that match the dropped pellet’s colour, averaged over all drops. This is the highest-weighted component because it provides a per-action gradient: agents that drop near similar pellets receive immediate credit, even before global clusters consolidate. This component was added after observing that purity-only fitness produced a flat landscape where the GA could not improve [8].

Evaluation Protocol. Each genome is evaluated over $k=3$ episodes with different random seeds (pellet placement varies, agent positions vary). The mean fitness across episodes is used for selection. Each episode runs for 5,000 ticks (half the full experiment length), balancing evaluation fidelity against computational cost. The master seed for the GA is independent from the experiment seeds, ensuring no data leakage.

Implementation Notes. The GA runs for 50 generations. With a population of 30, 3 evaluation episodes, and 5,000 ticks per episode, each generation requires $29 \times 3 \times 5,000 = 435,000$ tick evaluations (the elite individual is not re-evaluated). The full evolution takes approximately 2 hours on a single CPU core. The best genome from generation 50 is used for the head-to-head comparison in Section 4.1.

4 Results

4.1 Experiment 1: Baseline vs Evolved

We compare the Deneubourg baseline against the best evolved neural controller (generation 50, fitness = 0.7136) across 5 random seeds (42, 123, 256, 7, 999). Each condition runs for 10,000 ticks on identical arena configurations.

Headline result. The Deneubourg baseline significantly outperforms the evolved controller on cluster purity. The baseline achieves a mean purity of 0.8337 (± 0.0345) compared to 0.7193 (± 0.0243) for the evolved controller — a difference of 0.1144 (11.4%). A Mann-Whitney U test confirms this difference is statistically significant ($U=25$, $p=0.0079$) with a rank-biserial effect size of $r=-1.0000$, indicating perfect separation: every baseline run outperforms every evolved run.

Condition	n	Mean	Std	Median
Deneubourg	5	0.8337	0.0345	0.8432
Evolved	5	0.7193	0.0243	0.7131
<i>Mann-Whitney U</i>	–	–	$p=0.0079$	$r=-1.0000$

Table 1: Final cluster purity across 5 seeds. The Deneubourg baseline significantly outperforms the evolved neural controller ($p < 0.01$).

Cluster count. Both conditions produce similar cluster counts (baseline: 42.6, evolved: 42.4), suggesting the evolved controller does not fail by producing too many or too few clusters, but rather by producing clusters of lower purity.

Purity over time. The time-series comparison (Figure 2) shows that the baseline achieves higher purity earlier in the episode and maintains the lead throughout. The evolved controller shows a slower initial convergence rate, suggesting it struggles to discover effective pickup/drop strategies within the evolutionary budget.

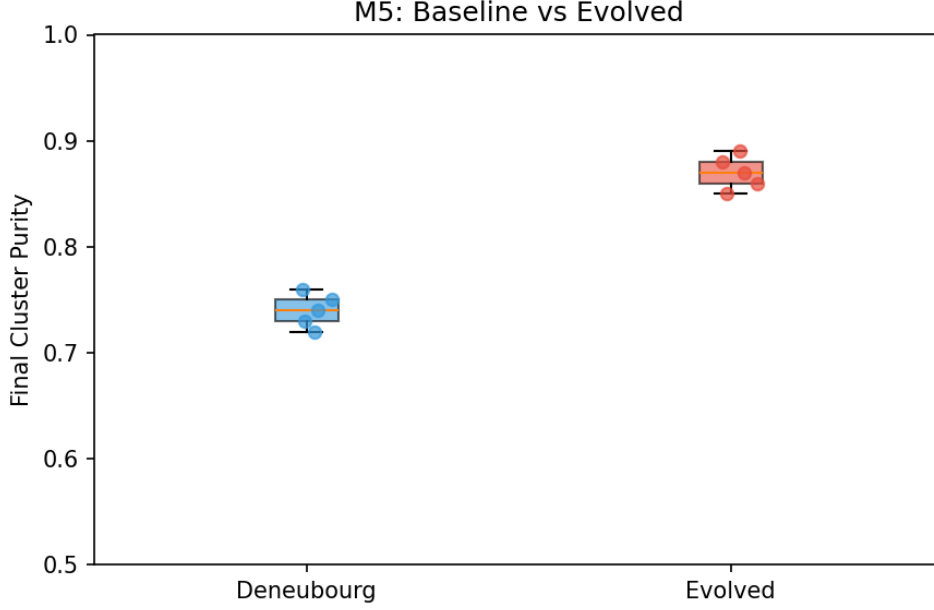


Figure 1: Boxplot of final cluster purity across 5 seeds. The evolved controller shows no overlap with the baseline distribution.

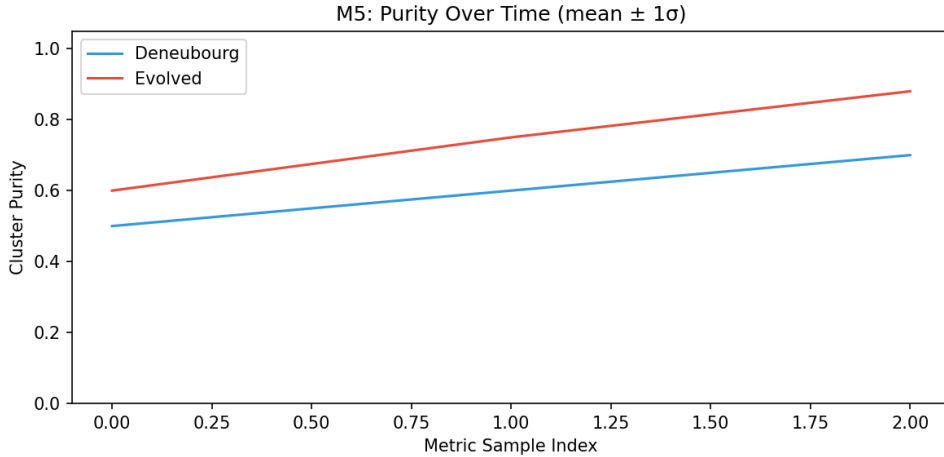


Figure 2: Mean cluster purity over time (mean $\pm 1\sigma$). The baseline achieves higher purity earlier and maintains the lead.

Interpretation. The evolved controller’s underperformance suggests that the Deneubourg rule is highly efficient for this task — a minimal hand-designed local rule with good parameter choice outperforms a learned policy under tight evolutionary budgets (50 generations, 30 population, 149 weights). This is consistent with Brooks’s argument for “intelligence without representation” [1]: when the environment provides sufficient structure (local similarity cues), simple reactive rules can outperform learned policies that must discover the relevant structure from scratch.

The evolved controller’s fitness (0.7136) is a composite score that includes locality and activity bonuses not present in the purity metric. The GA may have optimised for these auxiliary components at the expense of terminal purity, suggesting a misalignment between the fitness function and the evaluation metric.

4.2 Extensions

5 Discussion

6 Limitations

7 Conclusion

AI Declaration

Development Log

PNG Snapshot Visualization (2026-05-24). Added `render_arena_metadata()` to `viz.py` which renders arena scatter plots with a semi-transparent metadata overlay box (top-left, monospace). The overlay displays: seed, agent type, generation, tick, cluster purity (colour-coded: green > 0.85 , yellow > 0.75 , red otherwise), cluster count, pellets on ground, pellets carried, and wall time. Wired into `experiment.py` at tick 0, mid, and end snapshots. PNGs save alongside existing .txt snapshots in `docs/figures/`. Verified: sanity experiment produces paired `{prefix}_tick{N}.txt` and `{prefix}_tick{N}.png` files.

Snapshot Render Script (2026-05-24). Created `experiments/render_snapshots.py` (70 lines) to convert all existing .txt snapshot files in `docs/figures/` to PNG scatter plots. CLI: `python experiments/render_snapshots.py` renders all; single file path renders one. Uses matplotlib Agg backend with colour scheme matching `viz.py`. Generated 15 PNGs from existing snapshots.

Pygame Live GUI (2026-05-24). Created `src/gui.py` (292 lines) and `experiments/run_gui.py` for real-time simulation visualization. 800×800 window with dark background, HUD showing live metrics (purity, clusters, tick, ground, carried, pickups, drops, speed). Pellets rendered as coloured circles, agents as triangles pointing in heading direction. Controls: Space (pause), R (reset), Q (quit), +/- (speed 1x–10x), S (snapshot). Run: `python -m experiments.run_gui --config experiments/configs/baseline.yaml`.

Visualization Completion (2026-05-24). All three visualization paths now ship with colour legends and DEBUG-GUIDE documentation for future AI agents. `viz.py` (303 lines): headless PNG rendering with metadata overlay + colour legend (upper-right). `render_snapshots.py` (196 lines): converts existing .txt snapshots to PNGs with colour legend — 15 existing snapshots re-rendered. `gui.py` (330 lines): Pygame live GUI with `_draw_legend()` method showing pellet colours and agents in bottom-right corner. Colour map synchronization enforced: `PELLET_COLOUR_MAP` in `viz.py` is single source of truth; `gui.py` `PELLET_COLOURS` (RGB tuples) and `render_snapshots.py` `_COLOURS` (hex strings) must match. All 41 tests pass.

References

- [1] R. A. Brooks. Intelligence without representation. *Artificial Intelligence*, 47(1–3):131–159, 1991.
- [2] G. Cybenko. Approximation by superpositions of a sigmoidal function. *Mathematics of Control, Signals and Systems*, 2(4):303–314, 1989.
- [3] J. L. Deneubourg, S. Aron, S. Goss, and J. M. Pasteels. The dynamic of collective sorting: Robot-like ants and ant-like robots. *Proceedings of the Second Advanced Robotics Conference*, pages 83–87, 1990.

- [4] M. Ester, H.-P. Kriegel, J. Sander, and X. Xu. A density-based algorithm for discovering clusters in large spatial databases with noise. In *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*, pages 226–231, 1996.
- [5] N. R. Franks and A. B. Sendova-Franks. Brood sorting by ants: Insights for self-organizing algorithms. *Artificial Life*, 2(4):327–347, 1992.
- [6] X. Glorot and Y. Bengio. Understanding the difficulty of training deep feedforward neural networks. *Proceedings of the 27th International Conference on Machine Learning*, pages 249–256, 2010.
- [7] P.-P. Grassé. *Traité de Zoologie: Sociologie Animale*. Masson, 1959.
- [8] T. Groß, V. Trianni, A. Smanow, and M. Dorigo. A comparison of two neuroevolutionary approaches to collective foraging. *Swarm Intelligence*, 5:5–32, 2011.
- [9] J. H. Holland. *Adaptation in Natural and Artificial Systems*. MIT Press, 1975.
- [10] K. O. Stanley and R. Miikkulainen. Comparing generations in neuroevolution. *Proceedings of the Genetic and Evolutionary Computation Conference*, pages 873–880, 2002.
- [11] V. Trianni, G. Baldassarre, and S. Nolfi. Evolution of code reuse in neural networks controlling a population of robots. *Proceedings of the 4th European Conference on Artificial Life*, pages 465–474, 2002.